

NAE: Normalizing AutoEncoder

Muhammad Abdur Rafae*, Niels Landwehr

University of Hildesheim
rafaem/landwehr@uni-hildesheim.de

Abstract

We consider the setting of Normalizing flows with approximate inverses—an established paradigm spanning both full-dimensional ($d = D$) and bottleneck ($d < D$) settings—and group these models under the term *flow autoencoders*. We present a theoretical investigation into their training dynamics and prove that the proposed loss used by existing approaches is suboptimal; specifically, both encoder and decoder surrogates must be optimized in alignment with reconstruction loss. Guided by these insights, we propose Normalizing Autoencoder (NAE), which employs a novel conditional loss that aligns the surrogate loss gradient with that of reconstruction loss, directly improving upon the current standard. Extensive experiments across molecule generation, tabular data, and image benchmarks demonstrate that NAE achieves state-of-the-art performance. Our work highlights the importance of loss alignment in flow autoencoders and establishes NAE as a powerful generative framework.

1 Introduction

Normalizing flows are a powerful class of generative models that combine tractable likelihood evaluation with flexible density estimation by learning an invertible transformation between a simple base distribution and a complex data distribution (Walton 2023). However, exact invertibility typically requires specialized architectures, such as coupling layers, autoregressive transformations, or continuous-time dynamics. These restrictions can limit architectural flexibility or introduce costly numerical integration during training and sampling.

Recent work relaxes these restrictions by parameterizing the forward and inverse mappings as separate encoder and decoder networks (Sorensen et al. 2023; Draxler et al. 2024; Sorensen et al. 2024). The resulting models combine the structure of autoencoders with the likelihood-based training of normalizing flows; we refer to this emerging model class as **flow autoencoders**. This paradigm encompasses both full-dimensional ($d = D$) and bottleneck ($d < D$) settings, with both settings relying on surrogate-gradient formulations for efficient likelihood optimization.

Despite their promising empirical performance, the training dynamics of flow autoencoders remain underexplored.

Prior work introduced two surrogate formulations for estimating log-determinant gradients: an encoder surrogate (which updates the encoder) and a decoder surrogate (which updates the decoder). Empirically, training with the encoder surrogate is stable, whereas the decoder surrogate often leads to instability. This raises two central questions: *Why do the surrogates exhibit different training behavior, and can we effectively leverage both surrogates?*

In this work, we address these questions by introducing the **Normalizing Autoencoder (NAE)**, a flexible generative framework built on the flow autoencoder paradigm. Our contributions are threefold:

- We analyze the training dynamics of flow autoencoders and demonstrate that both surrogates are essential for effective learning.
- We propose a conditional surrogate loss that dynamically selects the surrogate whose gradient is aligned with the gradient of the reconstruction loss.
- We evaluate NAE across graph-structured, tabular, and vision benchmarks, demonstrating state-of-the-art performance across three data modalities.

2 Related Works

Normalizing Flows. Traditional normalizing flows rely on invertible transformations with tractable Jacobian determinants. Early methods used coupling and autoregressive layers with triangular Jacobians, such as NICE (Dinh, Krueger, and Bengio 2015), RealNVP (Dinh, Sohl-Dickstein, and Bengio 2016), MAF (Papamakarios, Pavlakou, and Murray 2017), and Glow (Kingma and Dhariwal 2018). Subsequent advances introduced spline-based coupling functions (Durkan et al. 2019; Hong and Chun 2023), invertible residual networks with Lipschitz constraints (Behrmann et al. 2019; Chen et al. 2020), and continuous normalizing flows via neural ODEs (Grathwohl et al. 2018). While expressive, these approaches impose rigid architectural constraints or require costly numerical integration during sampling, limiting their scalability to high-dimensional problems.

Injective Flows. A limitation of standard normalizing flows is the requirement for equal-dimensional data and latent spaces. However, real-world data often reside on a low-dimensional manifold (Bengio, Courville, and Vincent 2014), making a strict equal-dimensional transforma-

*Corresponding author.

tion highly restrictive. To address this, injective flows modify the change-of-variables formula for rectangular Jacobians (Köthe 2023). Common strategies include adding noise and denoising to the manifold (Horvat and Pfister 2021), constraining off-manifold latent variances (Beitler, Sosnovik, and Smeulders 2021; Silvestri, Roos, and Ambrogioni 2023), or using a two-stage procedure where an autoencoder is trained first, followed by a normalizing flow on its latent space (Brehmer and Cranmer 2020; Kothari et al. 2021; Böhm and Seljak 2020). Other methods impose strict geometric restrictions, such as conformal (Ross and Cresswell 2021) or isometric (Cramer et al. 2023) embeddings, which limit expressivity. Rectangular Flows (Caterini et al. 2021) introduced iterative, unbiased estimators for the Jacobian log-determinant, but they remain computationally expensive.

Flow Autoencoders. Recent work relaxes exact architectural invertibility by using separately parameterized encoder and decoder networks, trained as approximate inverses through a reconstruction penalty (Sorrenson et al. 2023; Draxler et al. 2024; Sorrenson et al. 2024). These models retain likelihood-based training while permitting flexible, off-the-shelf architectures. To avoid expensive log-determinant computations for general networks, existing methods use encoder and decoder surrogates. Although the two surrogates approximate their corresponding log-determinant gradients, prior work reports stable training primarily when using encoder surrogate, whereas the decoder surrogate often leads to instability.

Our work provides the first theoretical analysis of the interaction between the surrogate and reconstruction gradients in flow autoencoders. We then introduce a conditional loss that dynamically uses both the encoder and decoder surrogates, unlocking the full potential of flow autoencoders and establishing NAE as an effective generative model.

3 Methodology

To get insights into the training behaviour of the surrogate losses, we first review the surrogate terms and then derive their relation to the reconstruction loss.

3.1 Flow Autoencoders and Approximate Inverses

Let $E_\theta : \mathbb{R}^D \rightarrow \mathbb{R}^d$ and $D_\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ denote the encoder and decoder, respectively. Together, they define the autoencoder

$$f_\psi(x) = D_\phi(E_\theta(x)), \quad \psi = (\theta, \phi).$$

Let J_θ denote the encoder Jacobian at x_i , and G_ϕ the decoder Jacobian at $z_i = E_\theta(x_i)$. When the encoder and decoder are exact inverses of each other and $d = D$, the change-of-variables formula gives two equivalent forms,

$$\begin{aligned} \log p_X(x_i) &= \log p_Z(z_i) + \log |\det J_\theta| \\ &= \log p_Z(z_i) - \log |\det G_\phi|. \end{aligned} \quad (1)$$

For injective flows ($d < D$), the induced density is defined on the decoder manifold, and the volume change is determined by the corresponding Gram matrix (Krantz and

Parks 2008). The two possible nll expressions become:

$$\begin{aligned} \text{nll}_E(x_i) &= -\log p_Z(z_i) - \frac{1}{2} \log \det(J_\theta J_\theta^\top), \\ \text{nll}_D(x_i) &= -\log p_Z(z_i) + \frac{1}{2} \log \det(G_\phi^\top G_\phi). \end{aligned} \quad (2)$$

Both expressions reduce to Eq. (1) when $d = D$ and the networks are exact inverses.

To relax the exact invertibility constraint, Sorrenson et al. (2023) trains the autoencoder by minimizing the negative log-likelihood (nll) under a reconstruction penalty that enforces approximate inversion. The final loss used to train the flow autoencoder is given by:

$$\mathcal{L} = \sum_{i=1}^n [\text{nll}(x_i) + \beta \|x_i - f_\psi(x_i)\|_2^2], \quad (3)$$

where $\beta \geq 0$ is a Lagrange multiplier controlling the reconstruction constraint and will be referred to as *reconstruction weight*. When β is sufficiently large, the encoder and decoder behave as approximate inverses and either nll term from Eq. (2) can, in principle, drive training.

3.2 Surrogate Gradient Estimation

For general encoder and decoder architectures, evaluating the log-determinants in Eq. (2) is computationally expensive. Under the approximate-inverse assumption, Sorrenson et al. (2023) approximates their parameter gradients as

$$\begin{aligned} \nabla(\frac{1}{2} \log \det(J_\theta J_\theta^\top)) &\approx \text{tr}((\nabla J_\theta) G_\phi), \\ \nabla(\frac{1}{2} \log \det(G_\phi^\top G_\phi)) &\approx \text{tr}(J_\theta (\nabla G_\phi)). \end{aligned} \quad (4)$$

This estimator was subsequently extended to the full-dimensional setting ($d = D$) by Draxler et al. (2024).

To calculate the trace, Sorrenson et al. (2023) applies the Hutchinson trace estimator in latent space, where the smaller matrix dimension generally reduces the estimator variance. The probe vectors v_k are chosen as the first K columns of a randomly sampled orthonormal matrix $V \in \mathbb{R}^{d \times d}$ and scaled by $\sqrt{d}$. Consequently, $v_k^\top v_k = d$ and $\mathbb{E}[v_k v_k^\top] = I_d$. When $d = D$, the two trace orderings have comparable variance, and the trace is therefore estimated in data space (Draxler et al. 2024).

In practice, the trace is estimated using K vector-Jacobian and Jacobian-vector products along with `stop_gradient` operator $\mathbf{SG}(\cdot)$, which can be efficiently calculated in standard autodiff libraries like PyTorch. The corresponding scalar surrogates are given as:

$$\begin{aligned} \text{surr}_E &= -\frac{1}{K} \sum_{k=1}^K v_k^\top J_\theta \mathbf{SG}(G_\phi v_k), \\ \text{surr}_D &= \frac{1}{K} \sum_{k=1}^K \mathbf{SG}(v_k^\top J_\theta) G_\phi v_k. \end{aligned}$$

Backpropagating through these expressions produces the gradient estimates in Eq. (4). We therefore optimize

$$\begin{aligned} \widetilde{\text{nll}}_E &= -\log p_Z(z_i) + \text{surr}_E, \\ \widetilde{\text{nll}}_D &= -\log p_Z(z_i) + \text{surr}_D. \end{aligned} \quad (5)$$

The first surrogate (surr_E) updates the encoder through J_θ , whereas the second (surr_D) updates the decoder through G_ϕ . We denote them as *Encoder* and *Decoder* surrogates, respectively.

3.3 Reconstruction Loss Implicitly Regularizes the Jacobian

Continuous normalizing flows typically require discrete data, such as pixel-valued images, to be dequantized before density estimation. More generally, local perturbations can be added to the inputs to smooth the empirical data distribution (Kim et al. 2020; Ho et al. 2019). We show that minimizing the reconstruction loss under such perturbations implicitly regularizes the autoencoder’s Jacobian.

Let $X = x_i + \epsilon$ denote a perturbed observation, where the centered perturbation satisfies

$$\mathbb{E}[\epsilon] = 0, \quad \mathbb{E}[\epsilon\epsilon^\top] = \Sigma_i.$$

Here, ϵ may represent dequantization noise or explicit local augmentation. It may also model natural local variation, provided that such variation is approximately centered at x_i . Let $M_i = G_\phi J_\theta \in \mathbb{R}^{D \times D}$ denote the composite Jacobian of $f = D_\phi \circ E_\theta$ at x_i . For sufficiently small perturbations, a first-order expansion gives

$$f(x_i + \epsilon) \approx f(x_i) + M_i \epsilon. \quad (6)$$

Writing $r_i = x_i - f(x_i)$ and $A_i = I_D - M_i$, the expected local reconstruction loss becomes

$$\begin{aligned} \mathcal{R}_i &:= \mathbb{E}_\epsilon [\|X - f(X)\|_2^2] \\ &\approx \mathbb{E}_\epsilon [\|r_i + A_i \epsilon\|_2^2] \\ &= \|r_i\|_2^2 + 2r_i^\top A_i \mathbb{E}[\epsilon] + \text{tr}(A_i^\top A_i \Sigma_i) \\ &= \|x_i - f(x_i)\|_2^2 + \|A_i \Sigma_i^{1/2}\|_F^2, \end{aligned} \quad (7)$$

where the cross term vanishes because $\mathbb{E}[\epsilon] = 0$. Thus, reconstruction training encourages M_i to act as the identity along directions represented by the local covariance Σ_i .

We follow the setting of Sorrenson et al. (2023), where isotropic noise is added to input samples during training, such that $\Sigma_i = \sigma^2 I_D$. Therefore, Eq. (7) reduces to

$$\mathcal{R}_i = \|x_i - f(x_i)\|_2^2 + \sigma^2 \|I_D - M_i\|_F^2. \quad (8)$$

Up to normalization, the overall reconstruction objective is the sum of these local expectations:

$$\mathcal{L}_{\text{rec}} = \sum_{i=1}^n \mathcal{R}_i. \quad (9)$$

Data-Space to Latent-Space Relation. The matrix M_i acts in the data space. Analogously, we define $N_i = J_\theta G_\phi \in \mathbb{R}^{d \times d}$, which acts in the latent space. For brevity, we omit the sample and parameter subscripts. By the cyclic invariance of the trace,

$$\text{tr}(M) = \text{tr}(GJ) = \text{tr}(JG) = \text{tr}(N). \quad (10)$$

Under approximate-inverse assumption, N is close to I_d , while M is close to a rank- d orthogonal projector P_d (when $d = D$, M is close to I_D):

$$M = P_d + E_D, \quad N = I_d + E_d.$$

Since $\|P_d\|_F^2 = \|I_d\|_F^2 = d$, defining $\delta = \|E_D\|_F + \|E_d\|_F$ and applying the Cauchy–Schwarz inequality yields

$$\|M\|_F^2 - \|N\|_F^2 \leq 2\sqrt{d}\delta + \delta^2. \quad (11)$$

Thus, for small residuals, $\|M\|_F^2 \approx \|N\|_F^2$. Moreover,

$$\begin{aligned} \|I_D - M\|_F^2 &= D - 2\text{tr}(M) + \|M\|_F^2, \\ \|I_d - N\|_F^2 &= d - 2\text{tr}(N) + \|N\|_F^2. \end{aligned} \quad (12)$$

Combining these identities with Eq. (10) and Eq. (11) yields

$$\|I_D - M\|_F^2 \approx \|I_d - N\|_F^2 + D - d. \quad (13)$$

When J and G are exact Moore–Penrose inverses, the relation is exact. As D and d are constants, minimizing $\mathcal{L}_{\text{rec}}$ (Eq. 9) is approximately equivalent to minimizing

$$\tilde{\mathcal{L}}_{\text{rec}} \approx \sum_{i=1}^n [\|x_i - f(x_i)\|_2^2 + \sigma^2 \|I_d - N\|_F^2]. \quad (14)$$

3.4 Conditional Surrogate Loss

Differentiating the squared deviation $\|I_d - N\|_F^2$ with respect to the model parameters reveals a direct connection to the surrogate losses:

$$\begin{aligned} \nabla \tilde{\mathcal{L}}_{\text{rec}} &\propto \nabla \|I_d - N\|_F^2 \\ &\propto \text{tr}[(N - I_d)^\top \nabla N] \\ &\quad \text{where } \nabla N = (\nabla J)G + J(\nabla G). \end{aligned} \quad (15)$$

We’ll use the scaled orthonormal matrix V of size $d \times d$ ($VV^\top = V^\top V = I_d$, which was introduced in Sec 3.2) and the cyclic invariance of the trace:

$$\begin{aligned} \nabla \tilde{\mathcal{L}}_{\text{rec}} &\propto \text{tr}[(N - I_d)^\top V V^\top (\nabla N) V V^\top] \\ &\propto \text{tr}[V^\top (N - I_d)^\top V V^\top (\nabla N) V] \\ &\propto \text{tr}[(V^\top (N - I_d) V)^\top (V^\top (\nabla N) V)] \\ &\propto \sum_{i=1}^d \sum_{j=1}^d [v_i^\top (N - I_d) v_j] \cdot [v_i^\top (\nabla N) v_j]. \end{aligned}$$

Finally, focusing on the term with equal indices ($i = j$),

$$\begin{aligned} \nabla \tilde{\mathcal{L}}_{\text{rec}ii} &\propto [v_i^\top (N - I_d) v_i] \cdot [v_i^\top (\nabla N) v_i] \\ &\propto c_{ii} \cdot [v_i^\top (\nabla J) G v_i + v_i^\top J (\nabla G) v_i] \\ &\propto c_{ii} \cdot [\mathbf{SG}(v_i^\top J) G v_i + v_i^\top J \mathbf{SG}(G v_i)]. \end{aligned}$$

Crucially, for a given probe vector v_i , the scalar prefactor $c_{ii} = \mathbf{SG}(v_i^\top (JG - I_d) v_i)$ determines the behaviour of the composite Jacobian in the subspace of the probe vector. When the Jacobian is locally expansive ($c_{ii} > 0$):

$$\nabla \tilde{\mathcal{L}}_{\text{rec}ii} \propto \underbrace{+v_i^\top J \mathbf{SG}(G v_i)}_{-\text{surr}_{E_i}} + \underbrace{\mathbf{SG}(v_i^\top J) G v_i}_{\text{surr}_{D_i}}.$$

Similarly, when the Jacobian is locally contractive ($c_{ii} < 0$):

$$\nabla \tilde{\mathcal{L}}_{\text{rec}ii} \propto \underbrace{-v_i^\top J \mathbf{SG}(G v_i)}_{\text{surr}_{E_i}} - \underbrace{\mathbf{SG}(v_i^\top J) G v_i}_{-\text{surr}_{D_i}}.$$

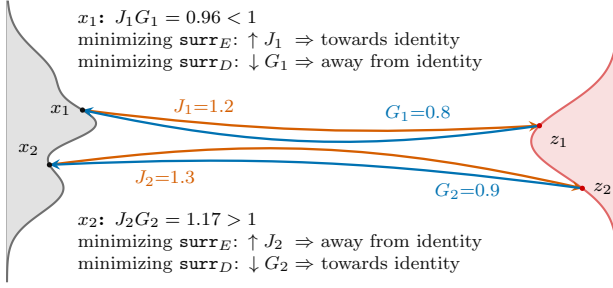


Figure 1: Two points x_1, x_2 are mapped from a data space X to a latent space Z via encoder and back via decoder, with scalar Jacobians J_i, G_i . For x_1 , we have $J_1 G_1 = 0.96 < 1$ (contractive): the reconstruction gradient is aligned with the surr_E , which pushes the composite Jacobian toward identity; but anti-aligned with the surr_D , which pushes the composite away from identity. The roles are reversed for x_2 , $J_2 G_2 = 1.17 > 1$ (expansive).

Within each probe subspace, one surrogate term (blue) aligns with the reconstruction-loss gradient, while the other (red) opposes it. Fig. 1 illustrates this for a 1D case.

We investigate this phenomenon further using a 4D toy dataset (Sec. 4.1). During training, we track the proportion of instances in which the surrogate and reconstruction-loss gradients align. The evolution of alignment throughout training is deferred to Appendix B.2, while Fig. 2 reports the final ratios. Models trained with surr_E maintain moderate alignment, whereas those trained with surr_D exhibit low alignment across most values of β , indicating that their gradients predominantly oppose the reconstruction-loss gradient. We hypothesize that this opposition causes training instability.

Based on this observation, we propose **Conditional Loss** that, for each probe, dynamically selects the surrogate term whose gradient is aligned with reconstruction-loss. Algorithm 1 provides pseudocode for the conditional loss in bottleneck setting. In full-dimensional setting, the trace is computed in data-space by swapping the JVP and VJP operations. Our method builds upon Sorrenson et al. (2023); Draxler et al. (2024) and introduces minimal computational overhead, specifically Lines 8–11.

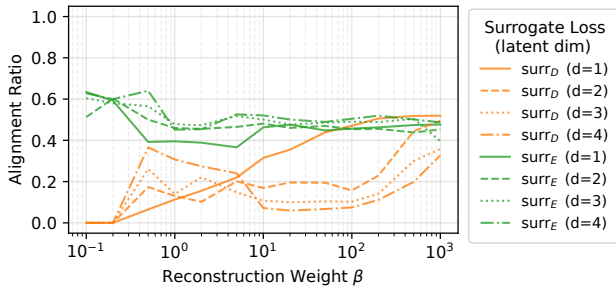


Figure 2: Alignment ratio (higher is better) averaged over 10 runs for models trained on 4D toy dataset with varying reconstruction weights β and latent dim d .

Algorithm 1: Conditional Loss

Input: Batch input x , encoder E_θ , decoder D_ϕ , latent dim d , number of Hutchinson probes K
Output: Surrogate loss $\mathcal{L}_{\text{cond}}$, encoder output z , decoder output $\hat{x}$

```

1: Let  $B$  = batch size
2: Generate Probes  $V$  of shape  $(K, B, d)$ 
3:  $\text{acc} \leftarrow 0$ 
4: for  $k = 1 \dots K$  do
5:    $z, J_k \leftarrow \text{VJP}(E_\theta, x, v_k) \{J_k = v_k^\top (\nabla E_\theta)\}$ 
6:    $\hat{x}, G_k \leftarrow \text{JVP}(D_\phi, z, v_k) \{G_k = (\nabla D_\phi) v_k\}$ 
7:    $\text{surr}_E \leftarrow -J_k \cdot \text{SG}(G_k) \{\text{detach } G_k\}$ 
8:    $\text{surr}_D \leftarrow \text{SG}(J_k) \cdot G_k \{\text{detach } J_k\}$ 
9:    $s_k \leftarrow \|v_k\|^2$ 
10:   $\text{mask} \leftarrow \mathbf{1}[\text{surr}_D > s_k] \{\text{binary indicator}\}$ 
11:   $\ell_k \leftarrow \text{SG}(\text{mask}) \cdot \text{surr}_D + (1 - \text{SG}(\text{mask})) \cdot \text{surr}_E$ 
12:   $\text{acc} \leftarrow \text{acc} + \ell_k$ 
13: end for
14:  $\mathcal{L}_{\text{cond}} \leftarrow \text{acc} / K \{\text{Surrogate loss to be minimized}\}$ 
15: return  $\mathcal{L}_{\text{cond}}, z, \hat{x}$ 

```

4 Experiments

We begin this section by evaluating our method on a synthetic toy dataset. We then evaluate the method first in the full-dimensional setting, and subsequently in the bottleneck setting. We use a single probe ($K = 1$) for Hutchinson trace estimation in all our experiments unless stated otherwise.

4.1 4D Toy Dataset

We construct a 4D synthetic dataset by concatenating samples from two 2D distributions: two moons and an eight-component GMM (data distribution shown in Fig. 3). We evaluate all latent dimensions $d \in \{1, 2, 3, 4\}$.

We train a simple MLP autoencoder using three methods: the **Decoder Surrogate**, **Encoder Surrogate**, and proposed **Conditional** loss. Each method is evaluated over a broad range of reconstruction weights β .

Fig. 3 presents the results for $d = 2$ and $d = 4$; complete results for $d = 1$ and $d = 3$, along with training details, are provided in Appendix B. We make the following observations:

- When training models across different β values, each method reaches the approximate-inverse regime at a different β value. For $d < D$, this transition value can be estimated using the elbow method heuristic on the reconstruction loss versus β graph.
- Training a model with a β higher than this transition value degrades the nll while yielding marginal improvements in reconstruction error.
- Since the proposed *Conditional* loss is aligned with the reconstruction objective by construction, compared to the other methods:
 - it requires a significantly smaller β to reach the approximate-inverse regime;
 - it achieves lower reconstruction error and nll within this regime.

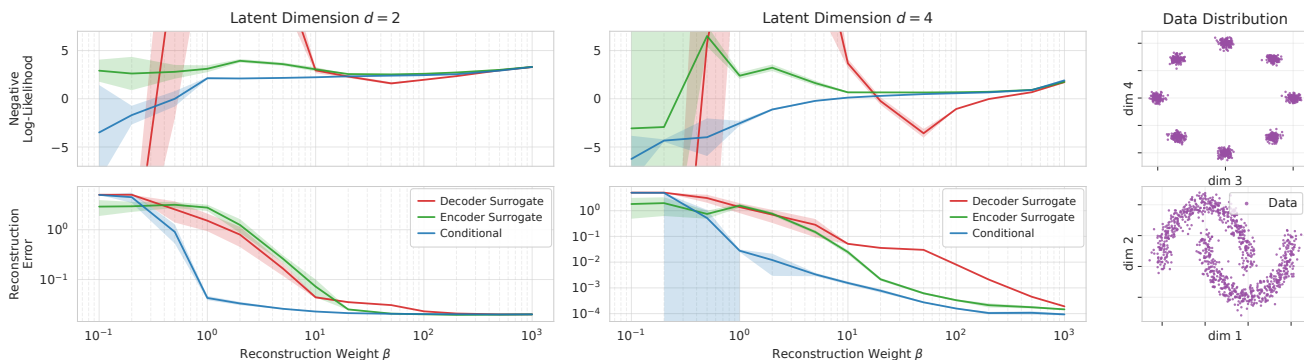


Figure 3: 4D Toy Dataset: *Left* and *middle* columns show test results for latent dimensions $d = 2$ and $d = 4$ respectively. The top row shows nll (lower is better), while the bottom row shows Reconstruction Error in log-scale (lower is better). Shaded regions indicate ± 1 standard deviation over 10 runs. *Right* column illustrates the toy data distribution; dim 1 and 2 corresponds to Two-moons and dim 3 and 4 corresponds to 8-GMM (eight Gaussian clusters evenly spaced along a circle).

4.2 Full-Dimensional Setting

In this section, we apply Normalizing Autoencoders (NAEs) to molecular generation. The objective is to model the joint probability distribution over atomic coordinates $x = (x_1, \dots, x_N) \in \mathbb{R}^{N \times n}$ for the DW4, LJ13, and LJ55 datasets, and over both coordinates and atomic properties (atom type and charge) for the QM9 dataset.

Equivariant Models Molecular systems exhibit Euclidean symmetry such that their physical properties are invariant to rotations and translations. Prior work (Köhler, Klein, and Noé 2020; Hooeboom et al. 2022) has shown that explicitly incorporating this inductive bias during training is more effective than relying on data augmentation. Therefore, we employ an $E(n)$ -Equivariant GNN (Satorras, Hooeboom, and Welling 2022) and adopt the exact model architecture, hyperparameters, and evaluation protocols from Draxler et al. (2024) without tuning. Implementation details are provided in Appendix C.2.

Boltzmann Generator We evaluate our model as a Boltzmann generator, which models the probability of a specific atomic configuration. The probability of a configuration depends on its energy via the Boltzmann distribution:

$$q(x) \propto e^{-\beta_t u(x)}, \quad (16)$$

where β_t is the inverse temperature of the system, and $u(x)$ is the energy function, which takes atomic positions $x = (x_1, \dots, x_N)$ as input.

We benchmark our method on **DW4** (four particles in 2D space interacting via a double-well potential), **LJ13** and **LJ55** (13 and 55 particles, respectively, in 3D space interacting via a Lennard-Jones potential). Further details of the DW and LJ potentials are provided in Appendix C.1. In all tasks, the total energy of the system is the sum of pairwise interactions $u(x) = \sum_{i,j} v(x_i, x_j)$.

For evaluation, we report the nll (under the Boltzmann distribution in Eq. 16) and the time required to generate samples. In Table 1, we compare against Equivariant ODE normalizing flow (E -NF) (Satorras, Hooeboom, and Welling

Method	DW4		LJ13		LJ55	
	nll	ST	nll	ST	nll	ST
E -NF	$1.72 \pm .01$	.024	$-16.28 \pm .04$	.27	OOM	
OT-FM	$1.70 \pm .02$	.034	$-16.54 \pm .03$	.77	$-88.45 \pm .04$	40.0
E -OT-FM	$1.68 \pm .01$	.033	$-16.70 \pm .12$	.72	$-89.27 \pm .04$	40.0
E -FFF	$1.68 \pm .01$	.026	$-17.09 \pm .16$	.11	$-88.72 \pm .16$	2.1
E -NAE	$1.39 \pm .01$	.026	$-17.95 \pm .16$	.11	$-92.32 \pm .09$	2.1

Table 1: Results on DW4, LJ13, LJ55 datasets: Average values with standard deviation over 3 runs for nll (lower is better) and ST (sampling time in ms, lower is better) are reported. **Bold** indicates the best performance. OOM: out-of-memory. Baseline results are taken from FFF.

2022); ODEs trained via optimal transport flow matching, both with equivariance (E -OT-FM) and without (OT-FM) (Klein, Krämer, and Noe 2023); and Equivariant Free-Form Flow (E -FFF), which employs the Encoder surrogate loss (Draxler et al. 2024).

Our method achieves lowest negative log-likelihood on all three datasets. Since NAE and FFF differ solely in their training objective while sharing the same architecture, their inference procedures are functionally equivalent. We therefore use the sampling times reported for FFF as direct proxies for those of NAE.

QM9 We also evaluate NAE on the QM9 dataset (Ramakrishnan et al. 2014), which contains molecules with variable numbers of atoms, with at most 29 atoms. The task requires the model to jointly model atomic coordinates and properties h_i (categorical atom type and ordinal charge). The network treats the atomic properties h_i as invariant under Euclidean transformations.

In Table 2, we compare against Equivariant Diffusion (E -DM) (Hooeboom et al. 2022); Equivariant ODE normalizing flow (E -NF) (Satorras, Hooeboom, and Welling 2022); and Equivariant Free-Form Flow (E -FFF) (Draxler et al. 2024). Our method achieves the best nll and Stable Sampling Time, while being competitive on molecular stability among non-diffusion methods.

Method	nll ($\downarrow$)	Molecular Stability ($\uparrow$)	Sampling Time	
			Raw ($\downarrow$)	Stable ($\downarrow$)
<i>E</i> -DM	-110.7	82.0%	1580.8	1970.6
<i>E</i> -NF	-59.7	4.9%	13.9	309.5
<i>E</i> -FFF	-76.2	8.7%	0.6	8.1
<i>E</i> -NAE	-120.8	9.3%	0.6	7.5

Table 2: Results on QM9 molecule Dataset. Validation nll (lower is better), molecular stability of generated samples (higher is better) and ST (sampling time in ms, lower is better) are reported. **Bold** indicates the best performance. Baseline results are taken from FFF.

Stable Sampling Time denotes the average time required to generate a stable molecule, including the time spent generating unstable samples that are subsequently discarded. While *E*-DM yields higher molecular stability, it does so at the cost of sampling time which is orders of magnitude slower. This highlights a key advantage of NAE over diffusion-based approaches, which are often preferred for generation tasks but suffer from iterative sampling overhead.

4.3 Bottleneck Setting

In this section, we first evaluate the generative performance of NAE on tabular datasets. We then assess our model on the Pythae image generation benchmark. For both tasks, we adopt the model architectures, hyperparameters, and evaluation protocols from Sorrenson et al. (2023). Finally, we compare our method against state-of-the-art injective flows. Implementation details are provided in Appendices D.2 and D.3.

Tabular Datasets We evaluate on four tabular datasets from Papamakarios, Pavlakou, and Murray (2017). We evaluate generative quality using the *FID-like* metric proposed by Caterini et al. (2021) and adopt model settings and hyperparameters from Sorrenson et al. (2023).

In Table 3, we compare against M-Flow (Brehmer and Cranmer 2020); Rectangular Flows (RF) (Caterini et al. 2021); Canonical Manifold Flows (CMF) (Flouris and Konukoglu 2023); and Free-Form Injective Flows (FIF) (Sorrenson et al. 2023). Our method achieves the best performance on three of the four datasets, achieving the lowest score on Power, HEPMASS, and MiniBooNE, while ranking second on Gas.

Method	Power	Gas	HEPMASS	MiniBooNE
M-Flow	0.258 $\pm$.045	0.219 $\pm$.016	0.741 $\pm$.052	1.650 $\pm$.105
RF	0.083 $\pm$.015	0.110$\pm$.021	0.779 $\pm$.191	1.001 $\pm$.051
CMF	0.053 $\pm$.005	0.373 $\pm$.065	0.574 $\pm$.044	1.508 $\pm$.082
FIF	0.041 $\pm$.007	0.281 $\pm$.031	0.541 $\pm$.034	0.598 $\pm$.024
NAE	0.013$\pm$.005	0.216 $\pm$.013	0.485$\pm$.025	0.583$\pm$.027

Table 3: Results on tabular datasets. Average FID scores (lower is better) and standard deviations across 5 runs are reported. **Bold** indicates the best performance, while underlined indicates the second-best. Baseline results are taken from their respective papers (except for M-flow, where the results are taken from CMF).

Model	ConvNet (33.5M Params)		ResNet (1.6M Params)	
	$\mathcal{N}$	GMM	$\mathcal{N}$	GMM
VAE	<u>54.8</u>	52.4	66.6	63.0
IWAE	55.7	52.7	67.6	64.1
VAE-lin NF	56.5	53.3	67.1	62.8
VAE-IAF	55.4	53.6	66.2	62.7
β -(TC) VAE	55.7	51.7	65.9	59.3
FactorVAE	53.8	52.4	66.4	63.3
InfoVAE	55.5	52.7	66.4	62.3
AAE	59.9	53.9	64.8	58.7
MSSSIM-VAE	124.3	124.3	119.0	119.2
Vanilla AE	327.7	55.4	275.0	57.4
WAE	64.6	51.7	67.1	57.7
VQVAE	306.9	51.6	140.3	57.9
RAE	86.1	52.5	168.7	58.3
FIF	56.9	47.3	62.3	55.0
NAE	57.6	36.9	60.4	53.4

Table 4: Pythae benchmark results on CelebA: FID scores (lower is better). **Bold** indicates the best performance, while underlined indicates the second-best. Results taken from FIF.

Pythae Benchmark Following Sorrenson et al. (2023), we benchmark NAE on the Pythae Benchmark (Chadebec, Vincent, and Allasoinniere 2022), which compares various autoencoder methods across two architectures and three datasets. All models are trained for the same number of iterations and evaluated using FID (Fréchet Inception Distance) and IS (Inception Score) under two latent prior distributions: a standard normal ($\mathcal{N}$) and a 10-component Gaussian mixture model (GMM) fit to the encoded training data. We report CelebA FID scores in Table 4, our method outperforms all baselines in three of the four settings and remains competitive in the fourth. Detailed results are deferred to Appendix D.3.

Higher-order derivatives. The models proposed in Pythae employ ReLU activations. Because the surrogate loss involves second-order derivatives, and $\text{ReLU}''(x) = 0$ almost everywhere (with autodiff implementations typically assigning it as zero at the origin), backpropagation through VJP/JVP operations discards all curvature information contributed by the activation functions. To address this limitation, we replace ReLU with SiLU, whose second derivative is nonzero, continuous, and smooth. In our experiments, this modification further improves the model performance. Owing to space constraints, we defer the corresponding results to Appendices D.2 and D.3.

Injective Models Finally, we compare our approach with three injective models on CelebA dataset: DNF (Horvat and Pfister 2021); Trumpets (Kothari et al. 2021); and FIF (Sorrenson et al. 2023). After observing overfitting with the architecture proposed in FIF, we reduced the model capacity rather than modifying the training pipeline. To establish the comparative computational budget on our hardware, we reproduced the reported FIF performance in 6.5 hours and allocated the same wall-clock time to our method. For evaluation, a 10-component GMM is also fit to the encoded training data. A GMM prior can be beneficial when the learned latent distribution contains disconnected, multimodal, or otherwise

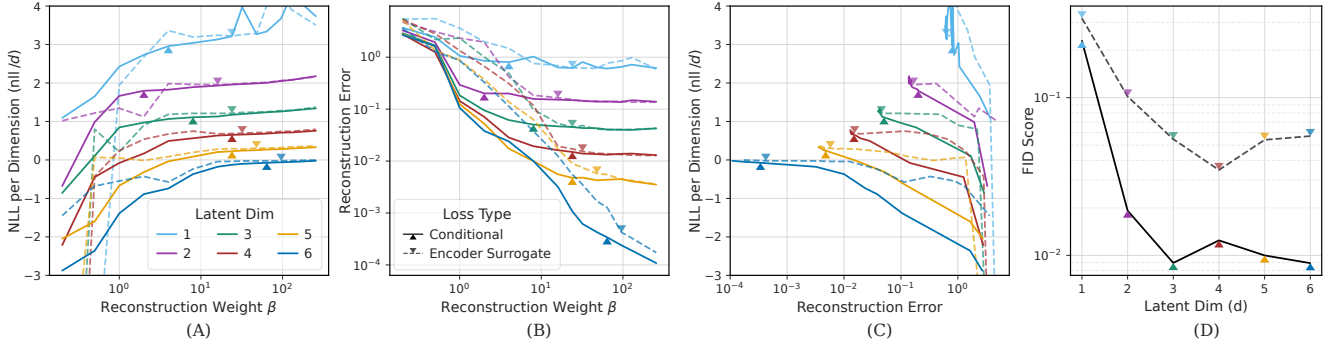


Figure 4: Ablation study on the Power dataset. Legend is shared across panels. Line Style denote loss type (solid for Conditional and dashed for Encoder Surrogate). Colours denote the latent dimension (d). Markers ($\blacktriangle$ for Conditional and $\blacktriangledown$ for Encoder Surrogate) indicate the β value that achieves the best FID-like metric for each configuration.

Model	# Params (M)	$\mathcal{N}$ sampler FID $\downarrow$ IS $\uparrow$	GMM sampler FID $\downarrow$ IS $\uparrow$	Δ FID
DNF	39.4	55.6 ± 0.59 1.9	52.7 ± 0.33 2.0	2.9
Trumpet	19.1	56.2 ± 1.39 1.8	47.7 ± 2.24 1.9	8.5
FIF	34.3	47.3 ± 1.39 1.7	37.4 ± 1.35 2.0	9.9
NAE	9.9	45.7 ± 0.43 2.1	41.2 ± 0.54 2.0	4.5

Table 5: Results for injective flows on CelebA. FID (lower is better) and IS (higher is better) are reported averaged over 3 runs with standard deviation. Number of parameters (in millions) of the model are also reported. The last column shows the absolute difference in FID between the samplers. Baseline results are taken from FIF.

non-Gaussian structure. A substantial improvement when sampling from the GMM rather than $\mathcal{N}$ suggests that its additional flexibility captures latent structure missed by the standard Gaussian. Conversely, a small improvement suggests that the learned latent representation is more regular and better aligned with the assumed Gaussian prior.

In Table 5, the strong performance of NAE with $\mathcal{N}$, together with the small FID gap between the two priors, therefore suggests that it learns a coherent latent representation that can be sampled effectively using a standard Gaussian. However, this small gap alone does not establish that the learned latent distribution is Gaussian. Further discussion is provided in Appendix D.3.

4.4 Ablation Study on Tabular Data

We conduct a controlled ablation study on real-world Power dataset (Papamakarios, Pavlakou, and Murray 2017) and compare our method with Encoder Surrogate loss across a range of reconstruction weights β and latent dimensions d . Implementation details are provided in Appendix F.

For each configuration, we report the mean over three independent runs of the negative log-likelihood per latent dimension (nll/d), reconstruction error, and FID.

Fig. 4(A,B) show that larger latent dimensions generally require larger values of β to reach the approximate-inverse regime. We hypothesize that increasing d alters the relative scales of the likelihood and reconstruction terms, thereby

requiring a larger reconstruction weight to balance the joint objective and ensure approximate inversion.

Fig. 4(C) further illustrates the trade-off between reconstruction error and nll as β varies: increasing β reduces the reconstruction error but increases nll. Since both quantities are minimized, configurations near the lower-left region of the plot are preferable.

Finally, Fig. 4(D) compares generative quality across different latent dimensions. The FID score decreases as d increases, reaches a minimum at an intermediate dimension, and then increases slightly. One possible explanation is that generative performance is optimized when the latent dimension coincides with the intrinsic dimensionality of the data manifold. A more thorough theoretical investigation of this phenomenon is left for future work.

5 Limitations and Conclusions

Limitations. Our analysis assumes locally smooth data densities and may therefore be less applicable when this assumption does not hold. Flow autoencoders also introduce an additional hyperparameter, the reconstruction weight β , whose appropriate scale may depend on the latent dimension and other model characteristics. For $d < D$, we recommend first training solely with the reconstruction loss to estimate the minimum attainable reconstruction error, then choosing β such that the training reconstruction error remains within a prescribed tolerance of this minimum.

Conclusions. This work presents a theoretical and empirical study of flow autoencoders. We establish that both encoder and decoder surrogate-loss terms are necessary for effective training. We introduce Normalizing AutoEncoder (NAE), which employs a novel conditional loss that ensures alignment of the surrogate loss with the reconstruction objective. We demonstrate NAE’s applicability for generative modeling across multiple domains. Our findings also open promising directions for future research, including the influence of activation functions on losses involving second-order derivatives, the extension of NAE to larger-scale datasets, and the exploration of more expressive encoder-decoder architectures for high-dimensional generative modeling.

Appendix

A Compute Details

All experiments were conducted on a server equipped with $7 \times$ *NVIDIA RTX A5000* GPUs, $1 \times$ *NVIDIA RTX PRO 6000* GPU, and two *AMD EPYC 7453* CPU processors.

B Toy Problem

B.1 Experimental Setup

All experiments used an MLP-based autoencoder architecture. The encoder and decoder each consisted of a five-layer MLP with a hidden dimension of 256 and SiLU (Swish) activations applied after each hidden layer. Neither network used skip connections. The weights were initialized using Xavier uniform initialization with a gain of $\sqrt{2}$. The input dimension was fixed at 4, while the latent dimension was specified separately for each experiment.

The dataset consisted of four-dimensional samples formed by concatenating two independent two-dimensional sources: (i) eight Gaussian clusters uniformly arranged on a circle of radius 2.0, each with a standard deviation of 0.1, and (ii) two interleaving half-circles generated using the scikit-learn `make_moons` function with a noise level of 0.1. A test set of 1,000 samples was generated using the same procedure.

Training was performed for 30,001 iterations with a batch size of 256. We used the Adam optimizer with an initial learning rate of 10^{-3} and a cosine annealing schedule spanning the full training duration. Gradients were clipped to a maximum norm of 1.0. The reconstruction loss was defined as the mean squared error (MSE) between the inputs and their reconstructions. A d -dimensional standard Gaussian distribution was used as the latent prior. The trace was estimated using a single Hutchinson probe.

For each method and β value, we conducted 10 independent runs. Random seeds were matched across methods for each run. Each model was trained on a single CPU core and required approximately 20 minutes.

B.2 Alignment Ratio

During training, we tracked the expansive ratio, defined as the proportion of instances in which the Jacobians are locally expansive within the probe subspace. Fig. 5 shows how this ratio evolves over the course of training with the Conditional loss. Across all latent dimensions d , the proportion of expansive Jacobians increases with the reconstruction weight β . Because a sufficiently large β is required to reach the approximate-inverse regime, a substantial proportion of Jacobians are expected to be expansive once this regime is reached.

For training with the Encoder and Decoder surrogates, we can compute the proportion of updates for which each surrogate is aligned with the reconstruction loss. Fig. 6 shows how these alignment ratios evolve over the course of training.

For the Decoder surrogate, the alignment ratio increases with β across all latent dimensions but rarely exceeds 40% when $d > 1$. This limited alignment with the reconstruction

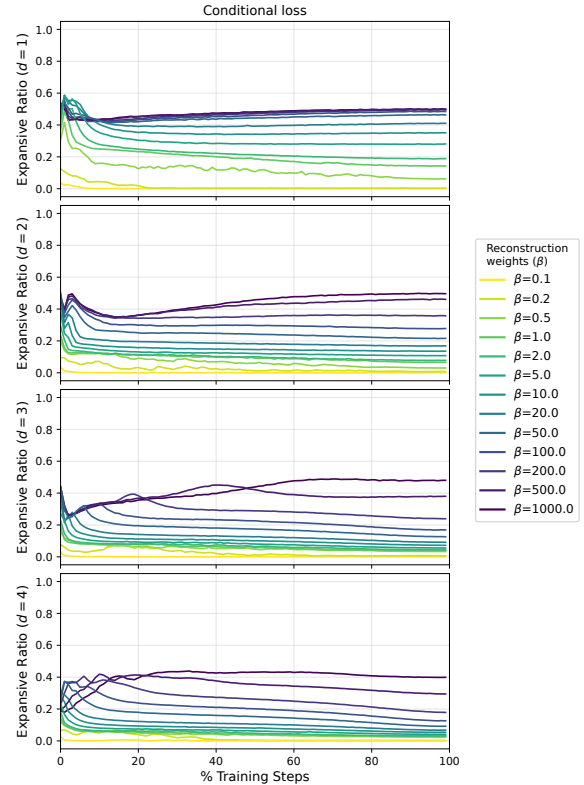


Figure 5: Expansive Ratio averaged over 10 runs for models trained using Conditional loss on the 4D toy dataset with varying reconstruction weights β and latent dim d .

objective explains the instability observed when training with the Decoder surrogate alone.

In contrast, the Encoder surrogate maintains an alignment ratio of 40–60% across all values of β and d , resulting in consistently more stable training than with the Decoder surrogate.

B.3 Extended Results

Figure 7 presents the results for $d = 1, 3$ using both nll and reconstruction error. These results are consistent with our observations for $d = 2, 4$: each method enters the approximate-inverse regime at a different value of β , with the Conditional loss reaching this regime at a significantly lower β than the other methods. Once in this regime, the Conditional loss consistently achieves superior reconstruction and nll performance.

We also observe that for $d = 1$, the Decoder surrogate performs better than the Encoder surrogate. We hypothesize that this occurs when the latent dimension is smaller than the intrinsic data dimensionality. A more thorough theoretical investigation of this phenomenon is left for future work.

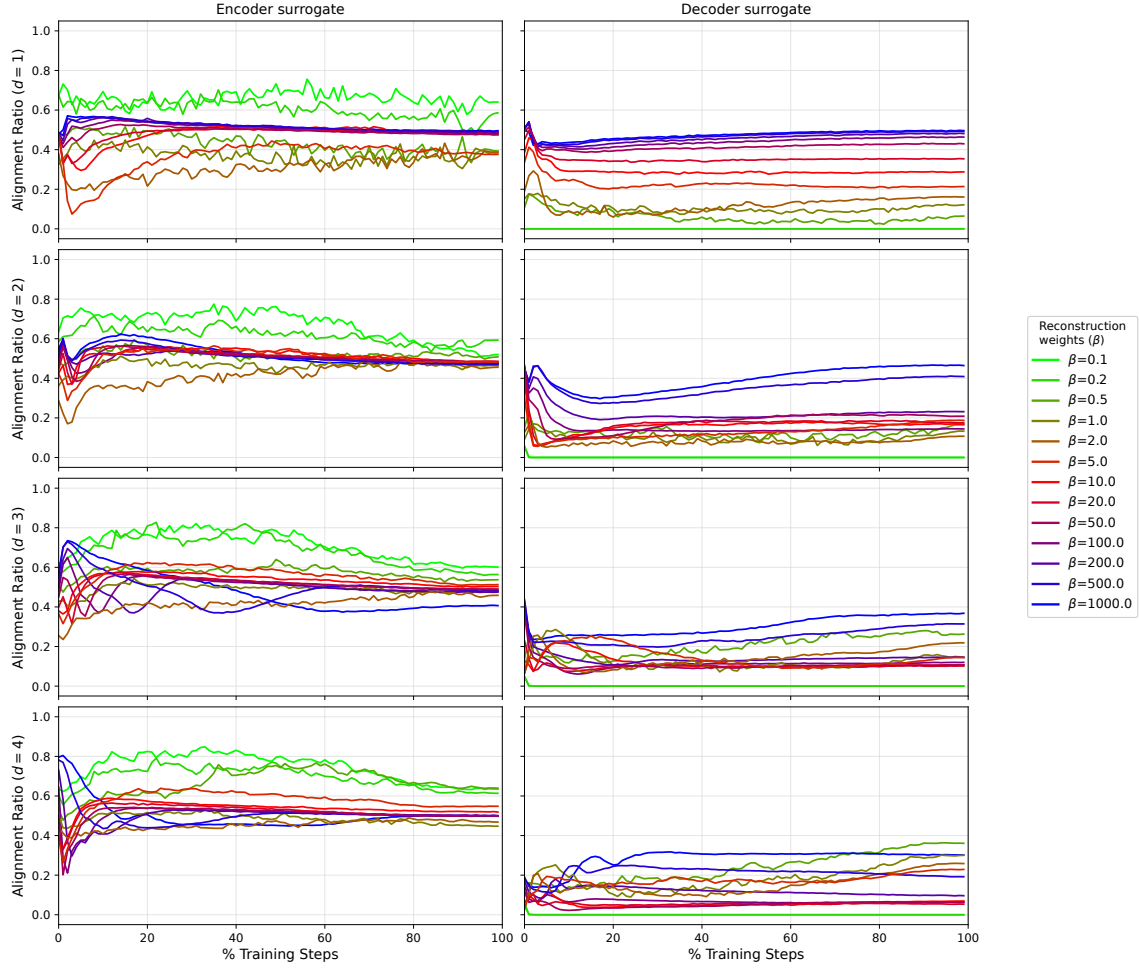


Figure 6: Alignment Ratios for Encoder and Decoder Surrogates. Alignment ratio (higher is better) averaged over 10 runs for models trained on 4D toy dataset with varying reconstruction weights β . Left and right columns represent Encoder and Decoder Surrogate losses and the rows represent the latent dim d .

C Full Dimension Setting

C.1 Datasets

DW4, LJ13 and LJ55 We evaluate our method on three standard Boltzmann-generator benchmarks: DW4, LJ13, and LJ55. These benchmarks model the distributions of atomic systems governed by pairwise interaction potentials. DW4 consists of four particles in two-dimensional space interacting through a double-well potential, whereas LJ13 and LJ55 consist of 13 and 55 particles, respectively, in three-dimensional space interacting through the Lennard-Jones potential.

The double-well potential v_{DW} and the Lennard-Jones potential v_{LJ} are defined as

$$v_{\text{DW}}(x_1, x_2) = \frac{1}{2\tau} \left(a(d - d_0) + b(d - d_0)^2 + c(d - d_0)^4 \right),$$

$$v_{\text{LJ}}(x_1, x_2) = \frac{\epsilon}{2\tau} \left[\left(\frac{r_m}{d} \right)^{12} - 2 \left(\frac{r_m}{d} \right)^6 \right],$$

where $d = \|x_1 - x_2\|$ denotes the Euclidean distance between

the two particles. We use the MCMC samples provided by (Klein, Krämer, and Noe 2023) as training data¹. We reserve 400k samples for validation and 500k samples for final testing. The inverse-temperature parameter β_t is set to 1.0 for all evaluations. Parameter values for both potentials are provided below:

- DW4: $a = 0, b = -4, c = 0.9, d_0 = 4$, and $\tau = 1$.
- LJ13 and LJ55: $r_m = 1, \epsilon = 1$, and $\tau = 1$.

QM9 The QM9 dataset (Ramakrishnan et al. 2014) contains molecules with varying numbers of atoms, with the largest containing 29 atoms. We use the same data splits as Garcia Satorras et al. (2021), 100K/18K/13K for training/validation/test respectively.

¹The datasets are available at https://osf.io/srqg7/?view_only=28deeba0845546fb96d1b2f355db0da5.

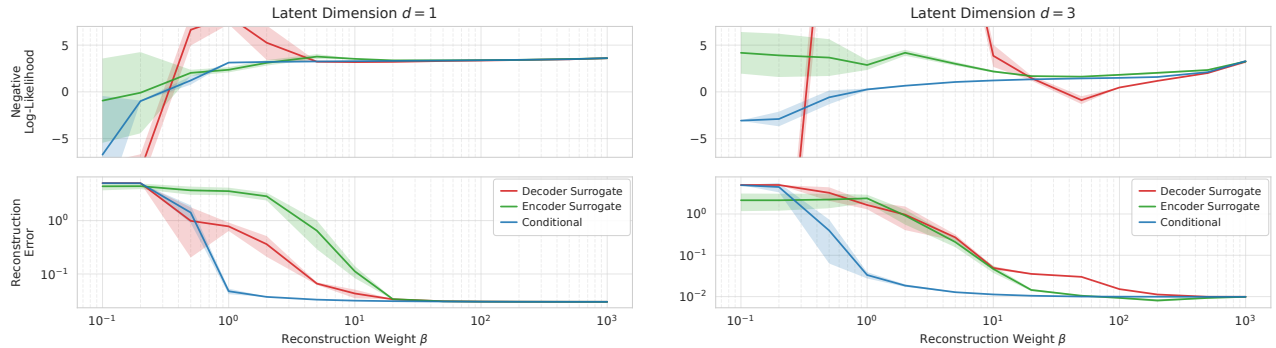


Figure 7: 4D Toy Dataset: Left and right columns show test results for models trained on latent dimensions $d = 1$ and $d = 3$, respectively. The top row shows Negative Log-Likelihood (nll) (lower is better), while the bottom row shows Reconstruction Loss in log-scale (lower is better). Shaded regions indicate ± 1 standard deviation over 10 runs.

	DW4	LJ13	LJ55	QM9
Layer count	20	8	10	16
Reconstruction weight β	10	200	500	2000
Learning rate	0.001	0.001	0.001	0.0001
Optimizer	Adam	Adam	Adam	Adam
Adam Betas	[0.9, 0.99]	[0.9, 0.99]	[0.9, 0.99]	[0.9, 0.99]
Learning rate scheduler	One cycle	–	– / Exponential $\gamma = 0.98$	– / Exponential $\gamma = 0.99$
Gradient clip	1	1	0.1	1
Batch size	256	256	48	48
Epochs	50	400	145 / 215	700 / 100
Training Time per epoch (mins)	5	2.5	11.5	23

Table 6: Hyperparameters used for the Molecular datasets. The format “A / B” specifies a two-step training procedure.

C.2 Model and Hyperparameters

We use the model architecture and hyperparameters proposed by Draxler et al. (2024) for *E*-FFF, with two exceptions. For **QM9**, we reduce the batch size from 64 to 48 because of GPU memory constraints. In addition, we set the Adam optimizer’s β_2 parameter to 0.99 rather than 0.999, as this improves training stability in our experiments. The complete hyperparameter configurations for all datasets are provided in Table 6.

We train the **LJ55** model on a single NVIDIA RTX PRO 6000 GPU and all other models on a single NVIDIA RTX A5000 GPU. The training time per epoch is reported in Table 6.

D Bottleneck Setting

D.1 Second-order derivative considerations

The models proposed in Sorrenson et al. (2023) for Tabular and Pythae benchmark, exclusively use ReLU activations. However, because the surrogate loss introduces second-order derivatives into the computation graph, backpropagating through the resulting VJP/JVP operations requires evaluating $\text{ReLU}''(x)$. Since $\text{ReLU}''(x) = 0$ almost everywhere and autodiff implementations typically assign a value of zero at the origin. The optimizer has access only to a simplified, locally linear representation of the network. Consequently, curvature information from the activation functions is discarded,

and the gradient provides no signal about how changes in the weights affect the network’s local sensitivity.

To address this limitation, we replace ReLU with SiLU, which is smooth and has a generally nonzero second derivative, thereby preserving the second-order gradient pathway. This substitution further improves performance, albeit at the cost of increased VRAM usage and training time because of the greater computational expense of evaluating SiLU and its derivatives. We leave a systematic study of the effects of different activation functions on Jacobian-based training to future work.

For a fair comparison with existing ReLU-based baselines, we evaluate our method using ReLU and report results obtained with SiLU separately to demonstrate the full potential of our approach.

D.2 Tabular Data

Experimental Details For the tabular-data experiments, we follow the setup of Caterini et al. (2021), using the same datasets, data splits, and latent-space dimensions. We adopt the model architecture proposed by Sorrenson et al. (2023): the encoder comprises a two-layer feed-forward network followed by a two-block ResNet, while the decoder mirrors this structure in reverse. All networks have a hidden width of 256 neurons.

We use the same training configuration, including a batch size of 512, isotropic Gaussian noise with a standard deviation of 0.01, $K = 1$ Hutchinson sample, gradient clipping

Hyperparameter	Power	Gas	HEPMASS	MiniBooNE
Latent dimension	3	2	10	21
Training epochs	15	30	85	875
Reconstruction weight β	10	10	10	5
Learning rate	0.001	0.001	0.0003	0.0001

Table 7: Dataset-dependent hyperparameters for tabular data experiments.

Method	Power	Gas	HEPMASS	MiniBooNE
M-Flow	0.258 $\pm$.045	0.219 $\pm$.016	0.741 $\pm$.052	1.650 $\pm$.105
RF	0.083 $\pm$.015	0.110$\pm$.021	0.779 $\pm$.191	1.001 $\pm$.051
RF*	0.074 $\pm$.012	0.283 $\pm$.031	0.628 $\pm$.046	1.622 $\pm$.121
CMF	0.053 $\pm$.005	0.373 $\pm$.065	0.574 $\pm$.044	1.508 $\pm$.082
FIF	0.041 $\pm$.007	0.281 $\pm$.031	0.541 $\pm$.034	0.598 $\pm$.024
NAE	0.013$\pm$.005	0.216 $\pm$.013	0.485$\pm$.025	0.583$\pm$.027
NAE-S	<i>0.011$\pm$.003</i>	0.252 $\pm$.006	<i>0.435$\pm$.016</i>	<i>0.534$\pm$.014</i>

Table 8: Results on tabular datasets. Average FID scores (lower is better) and standard deviations across 5 runs are reported. **Bold** indicates the best performance, while underlined indicates the second-best. M-Flow and **RF*** are the results reproduced by CMF. Results for NAE-S are included solely to demonstrate the potential of our method when paired with an appropriate activation function; they are not considered for direct comparison. However, when they outperform all other models, they are marked in *italics*.

with a maximum norm of 1.0, and the Adam optimizer with a one-cycle learning-rate scheduler. We do not use weight decay and set Adam’s β_2 parameter to 0.99 rather than 0.999, as we find that this improves training stability. The reconstruction weights β and learning rates are dataset-dependent and are listed in Table 7. For each dataset, we perform five independent runs. Each run takes approximately 15 minutes on a single *NVIDIA RTX A5000* GPU.

Extended Results We also evaluate all models using the SiLU activation function. For the Power, HEPMASS, and MiniBooNE datasets, replacing ReLU with SiLU yields further performance improvements.

For the Gas dataset, we attempted to reproduce the results of Caterini et al. (2021) using the authors’ original code but obtained a best FID of 0.183, which is notably worse than the reported value. Similar reproduction discrepancies have been documented by Flouris and Konukoglu (2023) and Sorrenson et al. (2023). For completeness, Table 8 also includes the Gas reproduction results reported by Flouris and Konukoglu (2023).

D.3 Pythae Benchmark

Experimental Details For the vision experiments, we follow the benchmark setup and training protocols of Chadebec, Vincent, and Allasounniere (2022). We adopt the same ConvNet and ResNet architectures used in the benchmark, as detailed in Tables 9 and 10. For MNIST and CIFAR-10, we train

each model for 100 epochs using the Adam optimizer with an initial learning rate of 10^{-4} , reserving the final 10,000 images of the training set for validation. For CelebA, we train for 50 epochs with an initial learning rate of 10^{-3} . All experiments use a batch size of 100, and the learning rate is halved when the validation loss does not improve for 10 consecutive epochs. Following the original benchmark, we perform a hyperparameter search over 10 configurations, select the model with the best validation FID, and report its test-set FID and Inception Score. Following Sorrenson et al. (2023), we search over five reconstruction weights, $\beta \in \{2, 5, 10, 20, 25\}$, and two numbers of Hutchinson probes, $K \in \{1, 2\}$.

Following Sorrenson et al. (2023), we exclude VAEGAN from the comparison because of its substantially higher training cost.

Extended Results Complete results for the Pythae benchmark are reported in Table 11. The large ConvNet-based models exhibit signs of overfitting, particularly on MNIST and CIFAR-10. We hypothesize that this behavior is partly attributable to the absence of data augmentation and dropout in the Pythae training pipeline. For consistency with the original benchmark, however, we leave the training protocol unchanged.

Alignment between Prior and Latent distribution. We also evaluate the models using the SiLU activation function and find that it further improves the performance of the ResNet-based architectures. Notably, for the ResNet-based NAE models with SiLU, sampling from the standard normal prior, $\mathcal{N}$, and from a 10-component GMM fitted to the encoded training data produces similar FID scores.

In generative models, fitting a GMM can improve sample quality when the learned latent distribution contains disconnected, multimodal, or otherwise non-Gaussian structure that cannot be adequately represented by a unimodal standard normal distribution. A substantial reduction in FID when switching from $\mathcal{N}$ to the fitted GMM may therefore indicate that the standard Gaussian fails to capture relevant structure in the learned latent distribution.

By contrast, the ResNet-based NAE models with SiLU obtain only a small reduction in FID when samples from $\mathcal{N}$ are replaced by samples from the fitted GMM. This limited benefit suggests that the learned latent representation is sufficiently regular and compatible with the assumed Gaussian prior, such that the additional flexibility of the GMM provides little practical advantage for generation. Together with the strong absolute performance obtained using $\mathcal{N}$, the small FID gap therefore suggests that NAE learns a coherent latent representation that can be sampled effectively using a standard normal distribution.

Nevertheless, this result provides only indirect evidence of prior-latent alignment and does not establish that the aggregated latent distribution is Gaussian. Similar FID scores may be observed even when deviations from Gaussianity are present but have a limited effect on the generated samples or are not adequately captured by FID. A direct characterization of the aggregated latent distribution and its alignment with the assumed prior is therefore left for future work.

Encoder	MNIST	CIFAR10	CELEBA
<i>Input</i>	(1, 28, 28)	(3, 32, 32)	(3, 64, 64)
Layer 1	Conv(128, 4, 2), BN, ReLU	Conv(128, 4, 2), BN, ReLU	Conv(128, 4, 2), BN, ReLU
Layer 2	Conv(256, 4, 2), BN, ReLU	Conv(256, 4, 2), BN, ReLU	Conv(256, 4, 2), BN, ReLU
Layer 3	Conv(512, 4, 2), BN, ReLU	Conv(512, 4, 2), BN, ReLU	Conv(512, 4, 2), BN, ReLU
Layer 4	Conv(1024, 4, 2), BN, ReLU	Conv(1024, 4, 2), BN, ReLU	Conv(1024, 4, 2), BN, ReLU
Layer 5	Linear(1024, 16)	Linear(4096, 256)	Linear(16384, 64)
Decoder	MNIST	CIFAR10	CELEBA
Layer 1	Linear(16, 16384)	Linear(256, 65536)	Linear(64, 65536)
Layer 2	ConvT(512, 3, 2), BN, ReLU	ConvT(512, 4, 2), BN, ReLU	ConvT(512, 5, 2), BN, ReLU
Layer 3	ConvT(256, 3, 2), BN, ReLU	ConvT(256, 4, 2), BN, ReLU	ConvT(256, 5, 2), BN, ReLU
Layer 4	Conv(1, 3, 2), Sigmoid	Conv(3, 4, 1), Sigmoid	ConvT(128, 5, 2), BN, ReLU
Layer 5	-	-	ConvT(3, 5, 1), Sigmoid
#Parameters	17.2M	39.4M	33.5M

Table 9: ConvNet, neural network architecture used for the convolutional networks, adapted from Sorrenson et al. (2023).

Encoder	MNIST	CIFAR10	CELEBA
<i>Input</i>	(1, 28, 28)	(3, 32, 32)	(3, 64, 64)
Layer 1	Conv(64, 4, 2)	Conv(64, 4, 2)	Conv(64, 4, 2)
Layer 2	Conv(128, 4, 2)	Conv(128, 4, 2)	Conv(128, 4, 2)
Layer 3	Conv(128, 3, 2)	Conv(128, 3, 1)	Conv(128, 3, 2)
Layer 4	ResBlock*	ResBlock*	Conv(128, 3, 2)
Layer 5	ResBlock*	ResBlock*	ResBlock*
Layer 6	Linear(2048, 16)	Linear(8192, 256)	ResBlock*
Layer 7	-	-	Linear(2048, 64)
Decoder	MNIST	CIFAR10	CELEBA
Layer 1	Linear(16, 2048)	Linear(256, 8192)	Linear(64, 2048)
Layer 2	ConvT(128, 3, 2)	ResBlock*	ConvT(128, 3, 2)
Layer 3	ResBlock*	ResBlock*	ResBlock*
Layer 4	ResBlock*, ReLU	ConvT(64, 4, 2)	ResBlock*
Layer 5	ConvT(64, 3, 2), ReLU	ConvT(3, 4, 2), Sigmoid	ConvT(128, 5, 2), Sigmoid
Layer 6	ConvT(1, 3, 2), Sigmoid	-	ConvT(64, 5, 2), Sigmoid
Layer 6	-	-	ConvT(3, 4, 2), Sigmoid
#Parameters	0.73M	4.8M	1.6M

Table 10: ResNet, neural network architecture used for the residual networks, adapted from Sorrenson et al. (2023). *The ResBlocks are composed of one Conv(32, 3, 1) followed by Conv(128, 1, 1) with ReLU.

E Injective Models

Experimental Details We follow the experimental setup of FIF (Sorrenson et al. 2023) for injective flows on CelebA. Rather than rerunning Trumpets (Kothari et al. 2021) and Denoising Normalizing Flows (DNF) (Horvat and Pfister 2021), we use the results reported by Sorrenson et al. (2023). The experiments in Sorrenson et al. (2023) use a five-hour training budget. To account for differences in hardware, we reproduce FIF on our hardware and obtain comparable performance after approximately 100 epochs, corresponding to 6.5 hours on a single *NVIDIA RTX A5000* GPU. We therefore adopt 6.5 hours as the wall-clock training budget for our method.

For the Normalizing Autoencoder (NAE), we initially use the same architecture as FIF but observe overfitting. To mitigate this, we halve the hidden dimensions of both the encoder and decoder and replace the activation function with SiLU. We train the model for 100 epochs using the Adam optimizer with a learning rate of 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.99$, a batch

size of 256, and a gradient-clipping threshold of 1.0. Architecture specifications for both FIF and NAE are provided in Table 12.

F Tabular Ablation

Experimental Details We perform a controlled ablation study on the Power dataset to characterize the sensitivity of the Conditional loss to the latent dimension d and reconstruction weight β . d determines the dimensionality of the learned representation, whereas β controls the relative contribution of the reconstruction term. Varying them jointly allows us to assess how the balance between likelihood optimization and approximate inversion changes with latent dimensionality and to compare this behaviour with that of the Encoder Surrogate loss.

We use the same model architecture, data preprocessing, optimizer, and training configuration as in the main tabular experiments, with ReLU activations. We sweep over all latent

Model (# Params)	Sampler	MNIST (17.2M)		ConvNet CIFAR10 (39.4M)		CELEBA (33.5M)		MNIST (0.73M)		ResNet CIFAR10 (4.8M)		CELEBA (1.6M)	
		FID	IS	FID	IS	FID	IS	FID	IS	FID	IS	FID	IS
NAE	$\mathcal{N}$	28.9	2.3	139.7	2.7	57.6	2.2	20.6	2.1	130.9	2.7	60.4	1.7
	GMM	10.2	2.2	112.1	3.4	36.9	2.1	12.2	2.1	122.9	3.3	53.4	1.8
NAE-SiLU	$\mathcal{N}$	28.3	2.3	125.2	3.1	54.2	2.1	13.5	2.1	<i>104.0</i>	3.7	<i>56.1</i>	1.7
	GMM	9.5	2.2	125.0	3.7	39.3	2.1	12.6	2.1	<i>103.6</i>	4.0	<i>53.3</i>	1.8
FIF	$\mathcal{N}$	23.8	2.2	121.0	3.0	56.9	2.1	19.5	2.1	132.6	2.9	62.3	1.7
	GMM	11.0	2.2	90.6	4.0	47.3	1.9	11.7	2.1	119.2	3.4	55.0	1.8
VAE	$\mathcal{N}$	28.5	2.1	241.0	2.2	<u>54.8</u>	1.9	31.3	2.0	181.7	2.5	66.6	1.6
	GMM	26.9	2.1	235.9	2.3	52.4	1.9	32.3	2.1	179.7	2.5	63.0	1.7
VAMP	VAMP	64.2	2.0	329.0	1.5	56.0	1.9	34.5	2.1	181.9	2.5	67.2	1.6
IWAE	$\mathcal{N}$	29.0	2.1	245.3	2.1	55.7	1.9	32.4	2.0	191.2	2.4	67.6	1.6
	GMM	28.4	2.1	241.2	2.1	52.7	1.9	34.4	2.1	188.8	2.4	64.1	1.7
VAE-lin NF	$\mathcal{N}$	29.3	2.1	240.3	2.1	56.5	1.9	32.5	2.0	185.5	2.4	67.1	1.6
	GMM	28.4	2.1	237.0	2.2	53.3	1.9	33.1	2.1	183.1	2.5	62.8	1.7
VAE-IAF	$\mathcal{N}$	27.5	2.1	236.0	2.2	55.4	1.9	30.6	2.0	183.6	2.5	66.2	1.6
	GMM	27.0	2.1	235.4	2.2	53.6	1.9	32.2	2.1	180.8	2.5	62.7	1.7
β -VAE	$\mathcal{N}$	21.4	2.1	115.4	3.6	56.1	1.9	19.1	2.0	124.9	3.4	65.9	1.6
	GMM	9.2	2.2	92.2	3.9	51.7	1.9	11.4	2.1	<u>112.6</u>	3.6	59.3	1.7
Dis β -VAE	$\mathcal{N}(0, 1)$	96.5	2.3	219.4	3.6	130.8	1.6	109.4	2.7	209.8	3.2	110.6	1.5
	GMM	192.7	2.3	300.8	1.8	94.0	1.5	98.7	2.1	202.3	2.6	83.2	1.6
	2-s sampler	236.1	1.5	371.2	1.0	167.9	1.0	250.8	1.1	349.0	1.2	161.0	1.2
	MAF sampler	191.8	2.2	300.6	1.8	94.3	1.5	98.3	2.1	200.6	2.6	82.6	1.7
β -TC VAE	$\mathcal{N}$	21.3	2.1	<u>116.6</u>	2.8	55.7	1.8	20.7	2.0	125.8	3.4	65.9	1.6
	GMM	11.6	2.2	<u>89.3</u>	4.1	51.8	1.9	13.3	2.1	106.5	3.7	59.3	1.7
FactorVAE	$\mathcal{N}$	27.0	2.1	236.5	2.2	53.8	1.9	31.0	2.0	185.4	2.5	66.4	1.7
	GMM	26.9	2.1	234.0	2.2	52.4	2.0	32.7	2.1	184.4	2.5	63.3	1.7
InfoVAE - RBF	$\mathcal{N}$	27.5	2.1	235.2	2.1	55.5	1.9	31.1	2.0	182.8	2.5	66.5	1.6
	GMM	26.7	2.1	230.4	2.2	52.7	1.9	32.3	2.1	179.5	2.5	62.8	1.7
InfoVAE - IMQ	$\mathcal{N}$	28.3	2.1	233.8	2.2	56.7	1.9	31.0	2.0	182.4	2.5	66.4	1.6
	GMM	27.7	2.1	231.9	2.2	53.7	1.9	32.8	2.1	180.7	2.6	62.3	1.7
AAE	$\mathcal{N}$	16.8	2.2	139.9	2.6	59.9	1.8	19.1	2.1	164.9	2.4	64.8	1.7
	GMM	9.3	2.2	92.1	3.8	53.9	2.0	11.1	2.1	118.5	3.5	58.7	1.8
MSSSIM-VAE	$\mathcal{N}$	26.7	2.2	279.9	1.7	124.3	1.3	28.0	2.1	254.2	1.7	119.0	1.3
	GMM	27.2	2.2	279.7	1.7	124.3	1.3	28.8	2.1	253.1	1.7	119.2	1.3
AE	$\mathcal{N}$	26.7	2.1	201.3	2.1	327.7	1.0	221.8	1.3	210.1	2.1	275.0	2.9
	GMM	9.3	2.2	97.3	3.6	55.4	2.0	<u>11.0</u>	2.1	120.7	3.4	<u>57.4</u>	1.8
WAE - RBF	$\mathcal{N}$	21.2	2.2	175.1	2.0	332.6	1.0	21.2	2.1	170.2	2.3	69.4	1.6
	GMM	9.2	2.2	97.1	3.6	55.0	2.0	11.2	2.1	120.3	3.4	58.3	1.7
WAE - IMQ	$\mathcal{N}$	<u>18.9</u>	2.2	164.4	2.2	64.6	1.7	20.3	2.1	150.7	2.5	67.1	1.6
	GMM	8.6	2.2	96.5	3.6	51.7	2.0	11.2	2.1	119.0	3.5	57.7	1.8
VQVAE	$\mathcal{N}$	28.2	2.0	152.2	2.0	306.9	1.0	170.7	1.6	195.7	1.9	140.3	2.2
	GMM	<u>9.1</u>	2.2	95.2	3.7	<u>51.6</u>	2.0	10.7	2.1	120.1	3.4	57.9	1.8
RAE - L2	$\mathcal{N}$	25.0	2.0	156.1	2.6	86.1	2.8	63.3	2.2	170.9	2.2	168.7	3.1
	GMM	<u>9.1</u>	2.2	85.3	3.9	55.2	1.9	11.5	2.1	122.5	3.4	58.3	1.8
RAE - GP	$\mathcal{N}$	27.1	2.1	196.8	2.1	86.1	2.4	61.5	2.2	229.1	2.0	201.9	3.1
	GMM	9.7	2.2	96.3	3.7	52.5	1.9	11.4	2.1	123.3	3.4	59.0	1.8
VAEGAN (not compared)	$\mathcal{N}$	8.7	2.2	199.5	2.2	39.7	1.9	12.8	2.2	198.7	2.2	122.8	2.0
	GMM	6.3	2.2	197.5	2.1	35.6	1.8	6.5	2.2	188.2	2.6	84.3	1.7

Table 11: Table taken from (Sorrenson et al. 2023) with our results added at the top. We report Inception Score (IS, higher is better) and Fréchet Inception Distance (FID, lower is better) computed with 10k samples on the test set. The best model per dataset and sampler is highlighted in **bold**, the second best is underlined. Results for NAE-SiLU are included solely to demonstrate the potential of our method when paired with an appropriate activation function; they are not considered for direct comparison. However, when they outperform all other models, they are marked in *italics*.

Encoder	NAE	FIF
<i>Input</i>	(3, 64, 64)	(3, 64, 64)
Layer 1	Conv(64, 4, 2), BN, SiLU	Conv(128, 4, 2), BN, ReLU
Layer 2	Conv(128, 4, 2), BN, SiLU	Conv(256, 4, 2), BN, ReLU
Layer 3	Conv(256, 4, 2), BN, SiLU	Conv(512, 4, 2), BN, ReLU
Layer 4	Conv(512, 4, 2), BN, SiLU	Conv(1024, 4, 2), BN, ReLU
Layer 5	Linear(8192, 64)	Linear(16384, 64)
Layer 6-9	4×ResBlock(128)*	4×ResBlock(256)*
Decoder	NAE	FIF
Layer 1-4	4×ResBlock(128)*	4×ResBlock(256)*
Layer 5	Linear(64, 32768)	Linear(64, 65536)
Layer 6	ConvT(256, 5, 2), BN, SiLU	ConvT(512, 5, 2), BN, ReLU
Layer 7	ConvT(128, 5, 2), BN, SiLU	ConvT(256, 5, 2), BN, ReLU
Layer 8	ConvT(64, 5, 2), BN, SiLU	ConvT(128, 5, 2), BN, ReLU
Layer 9	ConvT(3, 5, 1), Sigmoid	ConvT(3, 5, 1), Sigmoid
# Params	9.9M	34.3M

Table 12: ConvNet NAE and FIF (Sorrenson et al. 2023) models used for Celeba dataset for comparison to Trumpet and Denoising Normalizing Flow. *The ResBlocks(inner_dim) for both models are composed of Linear(latent_dim, inner_dim), SiLU, Linear(inner_dim, inner_dim), SiLU, Linear(inner_dim, latent_dim) with a skip connection.

dimensions $d \in \{1, 2, 3, 4, 5, 6\}$ and reconstruction weights:

$$\beta \in \{0.2, 0.5, 1, 2, 4, 8, 16, 24, 32, 48, 64, 96, 128, 256\}.$$

For each pair (d, β) , we train both objectives using three independent random seeds and evaluate them under the same protocol. Each experiment takes approximately 15 minutes on a single *NVIDIA RTX A5000* GPU.

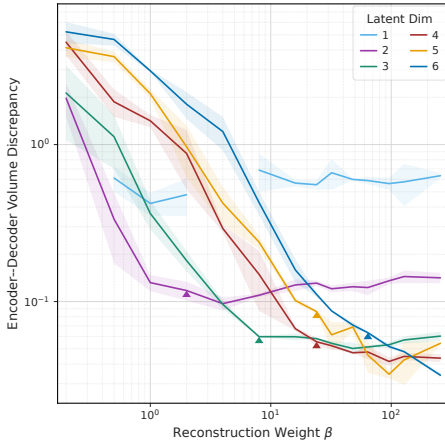


Figure 8: Encoder-decoder volume discrepancy for the Conditional loss. For each latent dimension d and reconstruction weight β , we compute the volume change of the encoder and decoder on the held-out test set using exact Jacobians. We report their absolute discrepancy normalized by the latent dimension, Δ_{vol}/d , for the final model. Curves show the mean over three independent runs, with colours indicating the latent dimension. Lower values indicate greater local geometric consistency between the encoder and decoder. Shaded regions indicate one standard deviation, and triangular markers denote the reconstruction weight that attains the lowest mean FID-like score for each latent dimension.

Extended Results To examine the approximate-inverse regime more directly, we measure the consistency between the local volume changes induced by the encoder and decoder. Specifically, we compute both quantities from exact Jacobians on the test set and report their absolute discrepancy normalized by the latent dimension. If the encoder and decoder are approximate inverses, their local geometric descriptions should be consistent, resulting in a small volume discrepancy.

As shown in Fig. 8, increasing the reconstruction weight generally reduces the volume discrepancy. For $d \geq 2$, the onset of the approximate-inverse regime identified by the reconstruction diagnostics coincides with the normalized discrepancy decreasing to approximately 10^{-1} or below. We interpret this value as an empirical diagnostic rather than a universal threshold, as a small volume discrepancy alone is insufficient to establish approximate inversion. For $d = 1$, the behaviour is erratic.

References

- Behrmann, J.; Grathwohl, W.; Chen, R. T. Q.; Duvenaud, D.; and Jacobsen, J.-H. 2019. Invertible Residual Networks. In *Proceedings of the 36th International Conference on Machine Learning*, 573–582. PMLR.
- Beitler, J. J.; Sosnovik, I.; and Smeulders, A. 2021. PIE: Pseudo-Invertible Encoder. arXiv:2111.00619.
- Bengio, Y.; Courville, A.; and Vincent, P. 2014. Representation Learning: A Review and New Perspectives. arXiv:1206.5538.
- Böhm, V.; and Seljak, U. 2020. Probabilistic Autoencoder. arXiv:2006.05479.
- Brehmer, J.; and Cranmer, K. 2020. Flows for Simultaneous Manifold Learning and Density Estimation. In *Advances in Neural Information Processing Systems*, volume 33, 442–453. Curran Associates, Inc.
- Caterini, A. L.; Loaiza-Ganem, G.; Pleiss, G.; and Cunningham, J. 2021. Rectangular Flows for Manifold Learning. In *Advances in Neural Information Processing Systems*, volume 34, 30228–30241. Curran Associates, Inc.
- Chadebec, C.; Vincent, L.; and Allasoinniere, S. 2022. Pythae: Unifying Generative Autoencoders in Python - A Benchmarking Use Case. In *Advances in Neural Information Processing Systems 35*, 21575–21589. New Orleans, Louisiana, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS). ISBN 978-1-7138-7108-8.
- Chen, R. T. Q.; Behrmann, J.; Duvenaud, D.; and Jacobsen, J.-H. 2020. Residual Flows for Invertible Generative Modeling. arXiv:1906.02735.
- Cramer, E.; Rauh, F.; Mitsos, A.; Tempone, R.; and Dahmen, M. 2023. Nonlinear Isometric Manifold Learning for Injective Normalizing Flows. arXiv:2203.03934.
- Dinh, L.; Krueger, D.; and Bengio, Y. 2015. NICE: Non-Linear Independent Components Estimation. In Bengio, Y.; and LeCun, Y., eds., *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Workshop Track Proceedings*. arXiv.
- Dinh, L.; Sohl-Dickstein, J.; and Bengio, S. 2016. Density Estimation Using Real NVP. arXiv:1605.08803.
- Draxler, F.; Sorrenson, P.; Zimmermann, L.; Rousselot, A.; and Köthe, U. 2024. Free-Form Flows: Make Any Architecture a Normalizing Flow. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, 2197–2205. PMLR.
- Durkan, C.; Bekasov, A.; Murray, I.; and Papamakarios, G. 2019. Cubic-Spline Flows. arXiv:1906.02145.
- Flouris, K.; and Konukoglu, E. 2023. Canonical Normalizing Flows for Manifold Learning. In *Advances in Neural Information Processing Systems*, volume 36, 27294–27314. New Orleans, Louisiana, USA: Curran Associates, Inc. ISBN 978-1-7138-9911-2.
- Garcia Satorras, V.; Hoogeboom, E.; Fuchs, F.; Posner, I.; and Welling, M. 2021. E(n) Equivariant Normalizing Flows. In Ranzato, M.; Beygelzimer, A.; Dauphin, Y.; Liang, P.; and Vaughan, J. W., eds., *Advances in Neural Information Processing Systems*, volume 34, 4181–4192. Curran Associates, Inc.
- Grathwohl, W.; Chen, R. T. Q.; Bettencourt, J.; Sutskever, I.; and Duvenaud, D. 2018. FFIORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models. *International Conference on Learning Representations*.
- Ho, J.; Chen, X.; Srinivas, A.; Duan, Y.; and Abbeel, P. 2019. Flow++: Improving Flow-Based Generative Models with Variational Dequantization and Architecture Design. arXiv:1902.00275.
- Hong, S.; and Chun, S. Y. 2023. Neural Diffeomorphic Non-uniform B-spline Flows. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(10): 12225–12233.
- Hoogeboom, E.; Satorras, V. G.; Vignac, C.; and Welling, M. 2022. Equivariant Diffusion for Molecule Generation in 3D. In *Proceedings of the 39th International Conference on Machine Learning*, 8867–8887. PMLR.
- Horvat, C.; and Pfister, J.-P. 2021. Denoising Normalizing Flow. In *Advances in Neural Information Processing Systems*, volume 34, 9099–9111. Curran Associates, Inc.
- Kim, H.; Lee, H.; Kang, W. H.; Lee, J. Y.; and Kim, N. S. 2020. SoftFlow: Probabilistic Framework for Normalizing Flow on Manifolds. arXiv:2006.04604.
- Kingma, D. P.; and Dhariwal, P. 2018. Glow: Generative Flow with Invertible 1x1 Convolutions. arXiv:1807.03039.
- Klein, L.; Krämer, A.; and Noe, F. 2023. Equivariant Flow Matching. In *Advances in Neural Information Processing Systems 36*, 59886–59910. New Orleans, Louisiana, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS). ISBN 978-1-7138-9911-2.
- Köhler, J.; Klein, L.; and Noé, F. 2020. Equivariant Flows: Exact Likelihood Generative Learning for Symmetric Densities. arXiv:2006.02425.
- Kothari, K.; Khorashadizadeh, A.; de Hoop, M.; and Dokmanić, I. 2021. Trumpets: Injective Flows for Inference and Inverse Problems. In *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, 1269–1278. PMLR.
- Köthe, U. 2023. A Review of Change of Variable Formulas for Generative Modeling. arXiv:2308.02652.
- Krantz, S. G.; and Parks, H. R. 2008. *Geometric Integration Theory*. Boston: Birkhäuser Boston. ISBN 978-0-8176-4676-9 978-0-8176-4679-0.
- Papamakarios, G.; Pavlakou, T.; and Murray, I. 2017. Masked Autoregressive Flow for Density Estimation. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Ramakrishnan, R.; Dral, P. O.; Rupp, M.; and von Lilienfeld, O. A. 2014. Quantum Chemistry Structures and Properties of 134 Kilo Molecules. *Scientific Data*, 1(1): 140022.
- Ross, B. L.; and Cresswell, J. C. 2021. Tractable Density Estimation on Learned Manifolds with Conformal Embedding Flows. arXiv:2106.05275.
- Satorras, V. G.; Hoogeboom, E.; and Welling, M. 2022. E(n) Equivariant Graph Neural Networks. arXiv:2102.09844.
- Silvestri, G.; Roos, D.; and Ambrogioni, L. 2023. Deterministic Training of Generative Autoencoders Using Invertible Layers. arXiv:2205.09546.

Sorrenson, P.; Draxler, F.; Rousselot, A.; Hummerich, S.; and Köthe, U. 2024. Learning Distributions on Manifolds with Free-Form Flows. In *Advances in Neural Information Processing Systems*, volume 37, 66961–66994. Vancouver, BC, Canada: Curran Associates, Inc. ISBN 979-8-3313-1438-5.

Sorrenson, P.; Draxler, F.; Rousselot, A.; Hummerich, S.; Zimmermann, L.; and Köthe, U. 2023. Lifting Architectural Constraints of Injective Flows. *International Conference on Learning Representations*.

Walton, S. 2023. Isomorphism, Normalizing Flows, and Density Estimation: Preserving Relationships Between Data. volume Technical Report.